A Paired, Floor-Guarded Evaluation Protocol for INT8 and FP8 Object Detectors under Image Corruptions

Dinh Thuan Nguyen[a], Lam Phuong Nguyen[a], Vinh Huy Nguyen[b], Sy Vu Quang[c], Mohan Rajesh Elara[d] and Anh Vu Le[c,*]

[a] *Faculty of Electrical and Electronics Engineering, Ton Duc Thang University, Ho Chi Minh City, Vietnam*
[b] *Center of Visualization and Simulation, Duy Tan University, Da Nang, Vietnam*
[c] *Advanced Intelligent Technology Research Group, Faculty of Electrical and Electronics Engineering, Ton Duc Thang University, 19 Nguyen Huu Tho Street, Tan Phong Ward, District 7, Ho Chi Minh City 700000, Vietnam*
[d] *ROAR Lab, Engineering Product Development, Singapore University of Technology and Design, Singapore*

ARTICLE INFO

Keywords:
Object detection
Robustness evaluation
Post-training quantization
Image corruption
Paired evaluation
INT8
FP8

ABSTRACT

Reliable comparison of quantized object detectors under image degradation requires separating clean-image discrepancies from changes caused by degradation. We introduce a paired, floor-guarded evaluation protocol for executable TensorRT INT8 and FP8 treatments. Both treatments receive identical encoded bytes, and a four-cell interaction subtracts the matched-clean FP8–INT8 gap from the corrupted-input gap within common-image bootstrap samples. The interaction is descriptive, not causal, and is interpreted with matched-clean fidelity and absolute corrupted average precision (AP). The exploratory grid contains 144 conditions across four datasets, three YOLO11 capacities, four synthetic corruptions, and three severities. A separate post-selection evaluation uses final VOC/KITTI images excluded from checkpoint selection; additional analyses vary aggregation scale, corruption materialization, training and calibration seeds, class composition, runtime, and detector architecture. FP8 outperformed INT8 by 1.40 matched-clean AP points across all 12 dataset–model blocks. The balanced additive interaction was -0.02 AP points (95% interval $[-0.24, +0.15]$), whereas the VOC/KITTI final holdouts yielded -0.55 $[-0.78, -0.29]$. Clean adjustment produced 56 point-estimate sign reversals; only 17 adjusted intervals lay entirely below zero, while 39 crossed zero. A relative-retention sensitivity disagreed in sign with the additive interaction in 18 of 144 cells, exposing scale dependence. AP decreased in 282 of 288 corrupted arms, and cross-family stress tests exposed failure of a transferred INT8 recipe. No universal format ranking follows; clean fidelity, absolute corrupted performance, and scale-explicit interaction must be reported together.

1. Introduction

Reliable image understanding requires not only accurate recognition on nominal benchmark images but also transparent evaluation when acquisition, weather, processing, or deployment conditions change. Evaluation-protocol design can materially affect conclusions about complete vision systems: UA-DETRAC, for example, evaluates coupled detection–tracking systems rather than isolated components [1]. Object detectors are increasingly deployed on devices with limited memory, energy, and computing capacity. Post-training quantization (PTQ) converts a trained network to lower precision using calibration, clipping, or reconstruction rather than repeating end-to-end training [2–4]. This deployment setting is distinct from quantization-aware training methods such as Learned Step Size Quantization, which learn quantization parameters during training [5]. INT8 has an established integer-arithmetic inference path, including hardware-aware integer-only designs [3, 6]. FP8 representations trade mantissa precision for exponent range and have developed from early low-precision training formats to formats and PTQ procedures designed for modern accelerators [7–10]. Deployment images may also contain noise, blur, poor visibility, or compression artifacts. Such corruptions can reduce accuracy in image classification, dense prediction, and object detection [11–14]. Comparisons of INT8 and FP8 should therefore consider both clean and corrupted images.

Quantization and image corruption are usually evaluated with separate measures. Quantization studies report the loss from full precision to a quantized model on clean images, while corruption studies report the loss from clean to corrupted images. Corruption benchmarks have also normalized each model's mean corruption performance by its own clean performance through relative performance under corruption (rPC) [13]. This practice is useful for within-model fractional retention, but it does not isolate how the gap between two deployment treatments changes on the same corrupted bytes. A raw FP8–INT8 difference on corrupted images combines the existing clean difference with the change from clean to corrupted inputs and may therefore be incorrectly interpreted as evidence that one treatment is more robust.

A second gap concerns interpretation near an accuracy floor. Severe corruption can reduce the average

* Corresponding author.
E-mail addresses: nguyendinhthuan.st@tdtu.edu.vn (D.T. Nguyen), nguyenphuonglam.st@tdtu.edu.vn (L.P. Nguyen), nguyenvinhhuy@dtu.edu.vn (V.H. Nguyen), vuquangsy@tdtu.edu.vn (S.V. Quang), rajeshelara@sutd.edu.sg (M.R. Elara), and leanhvu@tdtu.edu.vn (A.V. Le).

precision (AP) of both engines and make their difference smaller, even when neither engine remains useful. The labels "INT8" and "FP8" also do not fully describe an executable detector. Its behavior depends on quantizer placement, calibration data, graph changes, precision coverage, runtime settings, preprocessing, and decoding. We refer to this complete configuration as a treatment. A sound evaluation must therefore distinguish clean accuracy, absolute corrupted accuracy, and the corruption-associated change in the FP8–INT8 difference, while describing the complete treatments being compared.

To address these gaps, we propose a paired evaluation protocol with an absolute-AP guardrail. Each clean or corrupted condition is created once, and both TensorRT treatments receive identical image bytes. The main measure is an additive four-cell interaction that subtracts the FP8–INT8 difference on clean images from the corresponding difference on corrupted images. It is an algebraic, descriptive comparison rather than a causal estimator. We estimate the interaction within common-image bootstrap samples to preserve dependence between treatments and report a relative-retention counterpart to expose conclusions that depend on the AP scale. We interpret both measures together with clean and corrupted AP so that convergence near a common failure floor is not described as robustness. The study covers four datasets, three YOLO11 model sizes, four synthetic corruption types, and three severity levels, which gives 144 direct comparison cells. Additional analyses examine final VOC and KITTI partitions, alternative weights, repeated corruption samples, training and calibration seeds, a fixed class set, runtime, and stress transfer to Real Time Detection Transformer (RT-DETR) and RetinaNet. The goal is to evaluate the recorded TensorRT treatments rather than claim a universal ranking of all INT8 and FP8 implementations.

This study makes four main contributions:

- We introduce a paired four-condition protocol that gives INT8 and FP8 treatments identical image bytes and isolates the corruption-associated change in their accuracy difference.
- We define an absolute-AP guardrail that separates clean fidelity, remaining corrupted accuracy, and interaction, preventing convergence at low AP from being interpreted as retained robustness.
- We quantify decision and scale sensitivity through point-estimate sign reversals and a relative-retention contrast that complements the additive AP-point interaction.
- We establish a layered validation design spanning held-out final images, corruption materializations, training and calibration seeds, class composition, runtime, and recipe-portability stress tests across detector architectures.

The remainder of this paper is organized as follows. Section 2 reviews research on PTQ, numerical formats, and corruption robustness. Section 3 defines the treatments, datasets, corruption process, evaluation measures, and validation design. Section 4 presents the clean accuracy, corrupted accuracy, interaction, runtime, and transfer results. Section 5 discusses their meaning, practical implications, and limitations. Section 6 summarizes the main findings.

2. Related work

2.1. Quantization after training for detection

PTQ methods estimate numerical ranges with representative data or data-free surrogates, and reduce quantization error through clipping, rounding, or local reconstruction [2, 15–19]. QDrop further regularizes PTQ reconstruction by randomly omitting activation quantization during optimization [20]. These methods provide the calibration and reconstruction foundations used by later task-specific PTQ procedures.

Transformer quantization extends PTQ beyond convolutional classifiers. PTQ4ViT, FQ-ViT, RepQ-ViT, and I-ViT address activation distributions, scale reparameterization, and integer-only execution in vision transformers [21–24]. HAWQ-V3 similarly demonstrates hardware-aware dyadic quantization for integer-only inference [6]. Although these studies do not evaluate corrupted object-detection inputs, they show that numerical representation and executable constraints depend on architecture.

Object detection is especially sensitive to quantization because its classification head, localization or regression head, feature pyramid, and imbalanced foreground and background responses must remain consistent. Training-based approaches such as Quantization Mimic, Fully Quantized Networks, and AQD established detector-specific low-precision design but require distillation, retraining, or quantization-aware optimization [25–27]. PTQ-specific detector methods instead adapt calibration or reconstruction objectives: PD-Quant uses prediction differences, DetPTQ uses a detection-output loss, Reg-PTQ specializes calibration and quantization for regression structures, and InlierQ suppresses task-irrelevant activation anomalies with a small unlabeled calibration set [28–31]. Detector-specific efficiency work also shows that deployment cost is not reducible to the backbone alone; anchor pruning targets the detection head and the number of candidate boxes that reach post-processing [32]. These methods primarily target clean-input accuracy or computational cost. Our study addresses the related problem of evaluating already constructed quantized detectors when their input images are corrupted.

2.2. INT8, FP8, and executable treatments

Inference with integer arithmetic is an established hardware path for efficient neural networks, and integer-only systems have been demonstrated for both convolutional and transformer architectures [3, 6, 33]. Early FP8 studies developed low-precision training formats and hybrid arithmetic schemes [7, 8]. Later work formalized practical 8-bit floating-point formats, compared numerical layouts, and studied the role of exponent bits and outliers in inference quantization [9, 34, 35]. FP8 PTQ can retain competitive accuracy across several model classes [10], while flexible-format studies show that the preferred 8-bit representation can vary across layers and architectures [36]. A direct FP8–INT8 inference study further emphasizes that accuracy and dedicated-hardware efficiency can favor different representations [37]. These findings motivate a direct comparison, but they also show that the nominal datatype is not a complete description of an engine. Quantizer coverage, scale resolution, calibration, graph changes, and unsupported operations define the actual treatment. We therefore inspect the quantize and dequantize (Q/DQ) structure of each graph and interpret all results within the recorded ModelOpt and TensorRT pipelines.

2.3. Corruption robustness and quantized models

Standard corruption benchmarks provide controlled transformations for studying how image classifiers respond to changes outside the original test distribution [11]. Subsequent analysis has shown that apparent corruption robustness can depend on the relationship between training augmentations and the corruptions used for evaluation [38]. These benchmarks provide a common vocabulary for noise, blur, visibility changes, and acquisition or processing artifacts, but conclusions from classification do not transfer directly to dense prediction. Semantic segmentation and object-detection studies report task-specific failure patterns under weather effects, sensor noise, blur, and camera-related distortions [12–14]. Michaelis et al. aggregate corrupted performance as mean performance under corruption (mPC) and normalize it by clean performance as $\mathrm{rPC} = \mathrm{mPC}/P_{\mathrm{clean}}$ [13]. This answers how much of one model's clean performance is retained. Our additive interaction instead asks how the *between-treatment* AP-point gap changes for paired images; a secondary retention contrast tests whether its sign is sensitive to normalization. Robust object-detection research has also begun to model perturbation intensity explicitly rather than evaluating only one fixed attack strength [39].

A detector must preserve both class predictions and bounding-box localization. Diagnostic frameworks decompose detector errors into localization, classification, duplicate, and background components, while imbalance studies show that foreground–background and category distributions also affect detector behavior [40–42]. Recent work has examined adverse-weather detection, test-time adaptation for detection, efficient edge localization, and small-object detection [43–46]. Lightweight detector design under challenging UAV imagery further illustrates the need to evaluate robustness and efficiency together [47]. Small objects are particularly sensitive because only a limited number of pixels support their class and box estimates [48]. Corruption can further reduce this visual evidence. Nevertheless, low AP for small objects or under severe corruption does not establish that one numerical format is more sensitive than another.

The broader test-time adaptation literature updates a deployed model or its statistics after distribution change. Tent minimizes prediction entropy, CoTTA addresses continual test-time adaptation, and EATA limits unnecessary updates and forgetting [49–51]. These studies provide adaptation context rather than direct evidence about fixed quantized object detectors. Research specifically connecting quantization and distribution change remains less developed. Karimov et al. examine quantized detectors under degraded inputs and show the importance of evaluation beyond clean images [52]. Test Time Adaptive Quantization (TTAQ) adjusts PTQ during continuous domain change [53], while Recti-Q seeks to improve out-of-distribution quantized perception by correcting feature representations [54]. Work on adversarial robustness also indicates that different quantization procedures can lead to apparently inconsistent conclusions when the underlying pipelines are not aligned [55]. Together, these studies motivate an evaluation that records the complete executable treatment, uses the same inputs across precision formats, and states exactly which difference is being measured. This requirement is central to our comparison because the nominal labels INT8 and FP8 alone do not describe calibration, graph transformation, precision coverage, runtime behavior, or decoding.

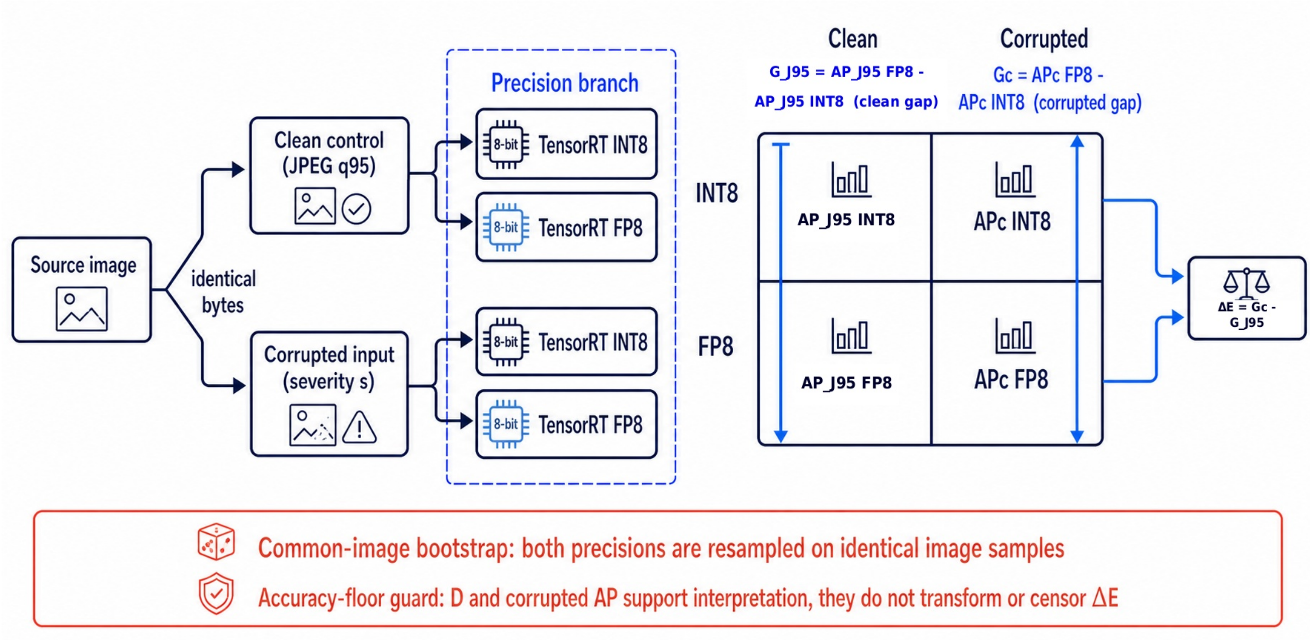

Figure 1: Paired evaluation protocol. A deterministic clean control encoded at JPEG quality 95 and each corrupted input are created before the precision branch. The four direct cells combine INT8 and FP8 with matched-clean and corrupted inputs; the FP32 engine is a separate diagnostic reference and cancels from the direct interaction. Common image samples preserve pairing. Matched-clean fidelity and absolute corrupted AP are checked before an interaction is interpreted.

2.4. Positioning of the present study

Benchmark and evaluation-protocol design is a substantive contribution when it changes the system-level question being measured [1]. Our work does not propose a new quantizer, corruption generator, detector, or clean-normalized robustness score; it provides a measurement protocol for executable vision engines. Three choices distinguish it from a raw corrupted-input comparison and from single-model rPC. First, clean and corrupted inputs use the same final codec, and both treatments receive identical bytes. Second, the clean FP8–INT8 gap is removed inside each common-image resample, directly targeting the change in a between-treatment difference. Third, the interaction is interpreted only after clean and absolute corrupted AP have been checked, and its additive scale is challenged by a relative-retention sensitivity. The resulting primary quantity is a paired two-by-two interaction contrast. Although it is algebraically analogous to a difference-in-differences expression, it invokes no causal identification assumptions and is not interpreted as a causal effect. The protocol can be used across architectures and runtimes, including cases in which a PTQ recipe does not transfer successfully.

3. Methodology

3.1. Problem Formulation

Figure 1 summarizes the study. The Microsoft Common Objects in Context (COCO) dataset uses the official Ultralytics YOLO11n, YOLO11m, and YOLO11x checkpoints. PASCAL Visual Object Classes (VOC), KITTI, and Tsinghua Tencent 100K (TT100K) use recorded `best.pt` checkpoints trained within the same YOLO11 family. Each checkpoint is exported to a set of executable TensorRT engines. We use 16 bit floating point (FP16) only as a basic consistency check. The main treatments are a TensorRT reference configured for 32 bit floating point (FP32), INT8 with entropy calibration, and FP8. We create the clean and corrupted images before selecting the precision treatment. Both quantized engines therefore receive the same ordered image identifiers and encoded bytes.

Throughout the study, INT8 and FP8 refer to the complete recorded ModelOpt and TensorRT treatments rather than datatype effects in isolation. Each treatment includes quantizer placement, Q/DQ coverage, the calibration list, scale resolution, graph changes, builder settings, runtime, preprocessing, and the decoder. The labels do not imply that every graph operation uses the named datatype. The study also does not estimate the best possible implementation of either format.

3.2. Estimands and interpretation order

Let d, m, q, b, and c denote the dataset, model, quantized treatment, size endpoint, and corruption condition. For the primary 144-cell comparison, $b =$ All denotes all-object AP; the small and large endpoints are used only for the separate size interaction. The symbol J95 denotes the corresponding matched-clean condition: the source image after deterministic Joint Photographic Experts Group (JPEG) encoding at quality 95 with chroma subsampling disabled and no named corruption. The label J95 is used instead of a bare "95" subscript to avoid confusion with AP evaluated at IoU 0.95. We calculate AP on its native $[0, 1]$ scale, multiply it by 100, and report the result in AP points.

The quantization gap for one quantized treatment is

$$Q_{d,m,q,b,c} = \mathrm{AP}_{d,m,\mathrm{FP32},b,c} - \mathrm{AP}_{d,m,q,b,c}. \tag{1}$$

The additional gap under corruption is

$$E_{d,m,q,b,c} = Q_{d,m,q,b,c} - Q_{d,m,q,b,\mathrm{J95}}. \tag{2}$$

Positive E means that corruption increases the gap between FP32 and the quantized treatment relative to clean images. Negative E means that the gap becomes smaller.

For the direct comparison, we define the FP8–INT8 gaps on clean and corrupted images as

$$G_{d,m,b,x} = \mathrm{AP}_{d,m,\mathrm{FP8},b,x} - \mathrm{AP}_{d,m,\mathrm{INT8},b,x}, \quad x \in \{\mathrm{J95}, c\}. \tag{3}$$

The primary interaction is

$$\begin{aligned}\Delta E_{d,m,b,c} &= E_{d,m,\mathrm{INT8},b,c} - E_{d,m,\mathrm{FP8},b,c} \\ &= G_{d,m,b,c} - G_{d,m,b,\mathrm{J95}}.\end{aligned} \tag{4}$$

The FP32 reference cancels in the algebra. Positive ΔE means that the FP8–INT8 AP gap is larger under corruption than on clean images. Negative ΔE means that the gap is smaller. Neither sign shows whether the absolute corrupted accuracy is useful in practice.

Because an additive AP-point interaction can depend on scale, we also report a secondary relative-retention sensitivity. For $q \in \{\mathrm{INT8}, \mathrm{FP8}\}$,

$$R_{d,m,q,b,c} = 100 \frac{\mathrm{AP}_{d,m,q,b,c}}{\mathrm{AP}_{d,m,q,b,\mathrm{J95}}}, \tag{5}$$

and

$$\Delta R_{d,m,b,c} = R_{d,m,\mathrm{FP8},b,c} - R_{d,m,\mathrm{INT8},b,c}. \tag{6}$$

Positive ΔR means that FP8 retains a larger percentage of its own matched-clean AP. Unlike ΔE, it is dimensionless and multiplicative; it is a sensitivity estimand rather than an interchangeable replacement for the primary additive contrast.

The signed clean-to-corrupted AP loss for precision treatment p is

$$D_{d,m,p,b,c} = \mathrm{AP}_{d,m,p,b,\mathrm{J95}} - \mathrm{AP}_{d,m,p,b,c}. \tag{7}$$

The direct interaction is also the difference between the two signed losses, $\Delta E_{d,m,b,c} = D_{d,m,\mathrm{INT8},b,c} - D_{d,m,\mathrm{FP8},b,c}$. Negative D denotes an AP gain after materialization rather than an absolute value. This identity provides an intuitive reading of the interaction while Eq. (4) preserves its paired four-cell construction. The contrast is conditional on the fixed benchmark treatments and does not require parallel-trends or other causal identification assumptions. "Floor-guarded" denotes a reporting rule: corrupted AP, D, and low-AP inventories accompany ΔE. It is not a statistical floor detector, a censoring correction, or a pass/fail threshold.

For COCO, VOC, and KITTI, the size interaction is

$$\Psi_{d,m,q,c} = E_{d,m,q,S,c} - E_{d,m,q,L,c}, \tag{8}$$

with the direct contrast $\Delta\Psi = \Psi_{\mathrm{INT8}} - \Psi_{\mathrm{FP8}}$. TT100K uses a similar contrast based on the original box height. We report the area and height endpoints as separate groups of measurements.

Table 1 gives the required order of interpretation. We first check clean accuracy, then the remaining absolute corrupted AP together with D, and only then interpret the signed interaction and its scale sensitivity. A favorable interaction does not by itself establish adequate absolute accuracy. Efficiency is considered only after these diagnostics.

3.3. Finite grid target and evidence layers

The exploratory study contains 4 datasets $\times 3$ YOLO11 model sizes $\times 4$ corruptions $\times 3$ severity levels $= 144$ direct cells. The balanced summary is the arithmetic mean over this defined finite grid,

$$\overline{\Delta E}_{\mathrm{grid}} = \frac{1}{144} \sum_{d,m,c} \Delta E_{d,m,\mathrm{All},c}. \tag{9}$$

It is not an effect weighted for an unspecified deployment population. We also report summaries weighted by image count and object count. In addition, we report summaries that omit one dataset or one corruption type at a time.

The evidence layers address different questions about validity (Table 2). The grid across four datasets is exploratory because the VOC2007 test set and the recorded KITTI validation partition were involved in checkpoint selection. A separate VOC and KITTI experiment uses retrained checkpoints and post-selection evaluation on final images excluded from checkpoint selection. It is the strongest holdout evidence for the stated comparison, but it covers only two datasets. Analyses of random seeds, corruption materializations, metric scale, a fixed class set, and other architectures are sensitivity analyses. They are not repeated estimates of one universal parameter.

Table 1
Measurement contract. All related comparisons use the same checkpoint, image order, image bytes, decoder, and image bootstrap sample.

Question	Quantity	Isolated comparison	Interpretation boundary
Clean accuracy	Q_{J95} or G_{J95}	Accuracy cost before any named corruption	Does not establish behavior under corruption.
Absolute corrupted accuracy	AP_c and D	Remaining task accuracy and the loss from clean to corrupted images	Shows whether both treatments converge at low corrupted AP.
Interaction for one treatment	$E = Q_c - Q_{\text{J95}}$	Corruption-associated change in the FP32 gap for one quantized treatment	Applies only to the recorded reference and treatment.
Direct treatment interaction	$\Delta E = G_c - G_{\text{J95}}$	Change in the INT8–FP8 treatment difference; FP32 cancels	Does not give a general treatment ranking; report it with clean and absolute AP.
Scale sensitivity	ΔR	Difference in the fractions of matched-clean AP retained by FP8 and INT8	Complements rather than replaces the additive ΔE.
Size interaction	$\Delta\Psi$	Difference between the treatment interactions for small and large objects	Keep area and height endpoints separate.
Efficiency	engine size and latency	Runtime of an accepted executable treatment	Engine latency is not total deployment latency.

Table 2
Evidence hierarchy and permitted interpretation.

Layer	Scope	Uncertainty or dependency examined	Permitted claim
Exploratory landscape	144 YOLO11 cells; 2,000 paired image samples	Uncertainty from the finite image sets and variation across factors	Conditional map and finite grid mean
Final holdout	VOC and KITTI; six retrained checkpoints; 72 cells	Sensitivity to evaluation after checkpoint selection; final-image reuse avoided	Post-selection estimate on final images excluded from selection
Metric scale	Primary and final-holdout AP components	Additive AP points versus fractional retention	Identifies scale-dependent signs; does not select a preferred scale
Training and calibration seeds	YOLO11m; three transfer datasets; severity 5	Recorded 3×3 seed crossing	Bounded descriptive seed sensitivity
Corruption realizations	36 base conditions; three frozen realizations	Stochastic materialization dependence	Bounded realization sensitivity
Fixed class set	One TT100K stress cell; 10,000 weighted samples	Sparse category composition	Sensitivity of one endpoint in one cell
Transfer across architectures	RT-DETR-L and RetinaNet; point estimates	Transfer of one fixed PTQ recipe	Evidence that can reveal failure, not format superiority

3.4. Datasets and size endpoints

Table 3 summarizes COCO val2017 [56], PASCAL VOC2007 test [57], the recorded KITTI validation split [58], and TT100K test [59]. We converted the VOC and KITTI annotations to COCO format and did not change them after conversion. The reported values are therefore COCO style measurements from converted annotations, not official VOC or KITTI challenge scores. The evaluator does not reproduce the difficult object rules from VOC or the `DontCare` and difficulty rules from KITTI.

Table 3
Frozen evaluation datasets. Counts refer to the converted annotations consumed by the evaluator.

Dataset	Split	Images	Objects	Declared/observed classes	Input px	Size strata
COCO	val2017	5000	36781	80/80	640	area: $< 32^2$ / $[32^2, 96^2)$ / $\geq 96^2$
VOC	VOC2007 test	4952	12032	20/20	640	area: $< 32^2$ / $[32^2, 96^2)$ / $\geq 96^2$
KITTI	validation	1496	8128	8/8	640	area: $< 32^2$ / $[32^2, 96^2)$ / $\geq 96^2$
TT100K	test	3067	8181	221/136	1280	height: XS/S/M/L/XL

TT100K contains 3,067 of the 3,071 referenced test images because four source files were unavailable. Its class map lists 221 categories, and 136 of them occur in the retained annotations. COCO, VOC, and KITTI group objects by their area A in the original image: small $A < 32^2$, medium $32^2 \leq A < 96^2$, and large $A \geq 96^2$ pixels. TT100K groups objects by the height h of the original bounding box: XS $0 \leq h < 12$, S $12 \leq h < 24$, M $24 \leq h < 48$, L $48 \leq h < 96$, and XL $h \geq 96$ pixels. The small endpoint combines XS and S, while the large endpoint combines L and XL.

3.5. Training, checkpoint selection, and holdout rerun

We evaluate YOLO11n, YOLO11m, and YOLO11x [60]. VOC and KITTI use 640×640 inputs; TT100K uses 1280×1280 so small signs retain more pixels. Every transfer model is initialized from the corresponding official COCO-pretrained Ultralytics checkpoint, with a frozen SHA-256 identity for each capacity. Exploratory training runs for 100 epochs with automatic mixed precision (AMP), seed 20260807, requested deterministic execution, eight workers, zero patience, and automatically selected optimizer and batch. Realized batches are 52/14/6 for VOC, 55/14/6 for KITTI, and 11/3/1 for TT100K (n/m/x). Ultralytics 8.4.95 selects momentum stochastic gradient descent (MuSGD; learning rate 0.01, momentum 0.9) for VOC, AdamW (learning rate 8.33×10^{-4}, betas 0.9/0.999) for KITTI, and AdamW (learning rate 4.4×10^{-5}, betas 0.9/0.999) for TT100K. Weight decay is configured as 5×10^{-4} and internally scaled by effective batch and accumulation; realized values and best epochs are reported in the Supplementary Information.

Augmentation uses hue, saturation, and value amplitudes of 0.015, 0.7, and 0.4, translation 0.1, scale 0.5, horizontal-flip probability 0.5, and mosaic probability 1.0, disabled for the final ten epochs. MixUp, CutMix, copy–paste, rotation, shear, and perspective are zero. For every dataset–capacity block, `best.pt` is the exponential-moving-average checkpoint maximizing selection-split COCO $\mathrm{AP}_{50:95}$ across the 100 epochs. The held-out rerun retains the same initialization identities, uses 100 epochs, batch 16, four workers, AMP, and seed 20260818; automatic selection gives MuSGD for VOC and AdamW for KITTI. COCO uses official checkpoints directly without retraining.

For the final rerun, all data partitions are frozen through a split registry generated with seed 20260818 before retraining. The VOC training set combines 8,218 images from train2007 and train2012. The 2,510 images in val2007 are used for checkpoint selection, whereas the 5,823 images in val2012 form the final evaluation set and are not involved in model selection. For KITTI, 1,496 of the 7,481 labeled images are first retained for selection. A deterministic, label-aware procedure then reserves a class-covering 20% subset of the remaining 5,985 images (1,197 images) for final evaluation; the other 4,788 images form the training partition. Each final partition contains every observed class. This construction protects class coverage but defines a conditional, designed KITTI holdout rather than a random sample of the official data distribution. Calibration samples are drawn only from the associated training partition. Confidence threshold, NMS, decoder, calibration procedure, and parity gates are frozen before metrics on the final images are calculated. Supplementary material reports image, object, and class counts for all partitions.

3.6. Executable precision treatments and calibration

We built the engines with TensorRT 11.1.0.106 [61] on an NVIDIA GeForce RTX 5090 and used a builder workspace of 4096 mebibytes (MiB). The historical exploratory reference did not explicitly disable TensorFloat 32 (TF32); “FP32” there denotes the implemented high-precision TensorRT reference rather than guaranteed strict IEEE FP32 execution for every kernel. This limitation does not affect direct ΔE, because FP32 cancels in Eq. (4). The final VOC/KITTI branch instead disables TF32 and gates source PyTorch against FP32 ONNX Runtime and FP32 TensorRT at 0.1 AP point, and FP32 against FP16 TensorRT at 1.0 AP point. The wider FP16 tolerance is only a gross consistency gate, not a claim of numerical equivalence.

Each calibration manifest contains a deterministic selection of 512 images from the training split and excludes all evaluation images. The rebuilt COCO engines use the identical calibration list for INT8 and FP8. The architecture-portability stress cases likewise use one fixed 512-image calibration set within each dataset. For the older exploratory transfer engines, the compact records do not retain enough build metadata to independently re-verify every paired calibration identity; those engines are therefore interpreted as complete recorded treatments rather than datatype-isolated effects, and calibration-list sensitivity is examined separately in Section 3.9. We produced quantized Open Neural Network Exchange (ONNX) graphs with the recorded ModelOpt `int8` and `fp8` modes and entropy calibration [62]. Inspection of all 24 quantized YOLO graphs confirmed integer zero-point tensors for INT8 Q/DQ nodes and `FLOAT8E4M3FN` zero-point tensors for FP8 Q/DQ nodes. Activation scales apply per tensor, weight scales use axis 0 and therefore apply per output channel, and all stored zero points are zero. Selected weights and activations are quantized. Sigmoid and operations outside Q/DQ regions use runtime-selected precision. The Supplementary Information reports graph-level Q/DQ coverage; these counts are not percentages of low-precision TensorRT kernels.

The recorded software environment used Linux kernel 7.0.0-28 with glibc 2.39, CUDA 12.8, PyTorch 2.11.0 + cu128, Ultralytics 8.4.95, ModelOpt 0.45.0 for the rebuilt COCO engines, ONNX 1.21.0, NumPy 2.4.4, Pillow 12.2.0, OpenCV 5.0.0, NVIDIA driver 610.43.02, and a batch size of one. The compact package does not retain a complete software/build manifest for the older exploratory transfer engines, so it should not be read as documenting every builder flag or package version for those artifacts. Their retained ONNX graphs nevertheless verify the Q/DQ datatype, scale axis, and zero point definitions.

3.7. Synthetic corruptions and execution controls

The suite contains four synthetic image corruptions. Gaussian noise draws an independent normal value for each RGB channel with zero mean and $\sigma \in \{8, 20, 40\}$ on the 0–255 scale, then clips to $[0, 255]$ before conversion to 8-bit pixels. Linear motion blur starts with a normalized horizontal line kernel of $\{5, 13, 23\}$ pixels, rotates it by an integer angle sampled uniformly from $\{0, \dots, 179\}$ degrees, renormalizes it, and applies OpenCV's `filter2D` with default border extrapolation. Fog creates a random grayscale field on a $\max(2, \lfloor h/32 \rfloor) \times \max(2, \lfloor w/32 \rfloor)$ grid, bicubically resizes it, applies Gaussian blur, and blends $I' = I(1 - sH) + 255sH$ with strength–radius pairs (s, r) of $(0.18, 6)$, $(0.38, 12)$, and $(0.62, 20)$. JPEG compression uses quality levels $\{70, 40, 15\}$ with chroma subsampling disabled. Severity labels 1, 3, and 5 order settings within each corruption but are not physically comparable across types. A SHA-256-derived seed binds dataset, image identifier, corruption, and severity. In the exploratory grid, each severity therefore receives an independently materialized random field or blur angle; severity summaries are descriptive profiles, not causal dose responses.

Every output is encoded deterministically as JPEG quality 95 with chroma subsampling disabled, and original dimensions are preserved. For the JPEG-corruption arm, the image is first compressed at the named quality and decoded; the result then passes through the same terminal JPEG-95 encoding as the other arms. The clean control skips the named corruption and receives only the terminal encoding. Each manifest records ordered image identifiers, file hashes, dimensions, and generator parameters. Corruptions are never generated inside an inference worker. We fix letterbox geometry, inverse mapping, a confidence threshold of 10^{-3}, class-wise NMS at IoU 0.7, and a decoder cap of 300 detections per image. COCO-style AP evaluation subsequently uses `maxDets` $= 100$. Each run record links the engine, decoder, annotations, class map, input manifest, image order, and prediction identity.

3.8. AP evaluation and paired uncertainty

We evaluate AP across IoU thresholds from 0.50 to 0.95 in steps of 0.05. The image is the resampling unit, following the nonparametric bootstrap principle of resampling observational units [63]. For each dataset, we reuse one deterministic schedule of 2,000 samples of image indices across every related clean, corrupted, INT8, and FP8 treatment. A repeated image position becomes a repeated evaluator entry with a unique identity within that bootstrap sample. This approach preserves image multiplicity and stable score order. The common schedule also preserves covariance that would be lost if the treatment summaries were bootstrapped separately.

Percentile intervals with two tails describe uncertainty from the finite evaluation images while the checkpoints, engines, calibration lists, decoder, corrupted images, and evaluation split remain fixed. The intervals do not include uncertainty from training seeds, calibration samples, engine builds, dataset selection, or a population of possible corruptions. Signs of intervals for individual cells are exploratory and not multiplicity-adjusted. We calculate the grid, area, and TT100K height summaries within common samples and use fixed balanced weights. Area and height endpoints are never pooled. Direct contrasts are calculated from unrounded cell values.

The decision-impact diagnostic compares the strict point-estimate signs of the raw corrupted gap G_c and the adjusted interaction ΔE; an exact zero is treated as a tie and not as a reversal. We additionally classify an adjusted percentile interval as below zero, above zero, or crossing zero. Magnitude filters at 0.25, 0.50, and 1.00 AP points are descriptive sensitivity thresholds. The relative-retention analysis in Eq. (6) reuses the same four AP components but is reported as a point-estimate scale sensitivity rather than an additional confidence interval.

Resampling TT100K can change the effective set of positive classes because categories with no positive examples are omitted from the AP average. We therefore perform a targeted sensitivity analysis for TT100K, YOLO11m, motion blur, and severity 5. It uses 10,000 Bayesian bootstrap samples with positive image weights [64] and fixes the category and endpoint set to the full annotation of 3,067 images. Each draw assigns independent exponential weights with unit rate to the images, normalizes the 3,067 weights to mean one, applies them to ground-truth positives and true/false-positive contributions, and reuses the same vector across all four treatment arms. This analysis defines a separate sensitivity measure and does not replace the standard image bootstrap.

3.9. Sensitivity designs

To examine sensitivity to corruption materialization, we evaluate YOLO11m on deterministic, label-aware, class-covering subsets of 512 images from VOC, KITTI, and TT100K. Three frozen seeds, 202608181, 202608182, and 202608183, generate independent realizations. Within an image–seed pair, the random field or transformation angle is held constant across severities while only magnitude changes; JPEG is deterministic. This branch therefore differs from the exploratory grid in four coupled ways: it fixes YOLO11m, excludes COCO, uses 512-image subsets, and nests stochastic fields across severity instead of materializing severities independently. It contains 36 base conditions and 108 realization cells. The nested summary resamples the three realization labels and applies paired image draws within each label. With only three labels, the resulting percentiles are a bounded sensitivity construction, not a population interval over possible corruptions.

We separately assess sensitivity to training and calibration choices using YOLO11m on VOC, KITTI, and TT100K at severity level 5. Training seeds 20260807, 20260813, and 20260814 are crossed with three deterministic calibration lists, each containing 512 training images and generated from the corresponding seeds. All other elements of the experimental procedure remain fixed, so variation is restricted to the training seed and the calibration-list seed. The resulting 3×3 factorial design yields 54 TensorRT engines and 108 direct comparison cells across the datasets and corruption types. The reported marginal means and ranges summarize these specific configurations; they are not treated as estimates of variance over a broader population of possible seeds.

Because data sampling, initialization, and other pipeline choices can contribute materially to benchmark variation, we keep the frozen variation sources visible rather than treating one run as universal evidence [65]. Finally, we evaluate whether aggregate conclusions depend on weighting or on a particular experimental group. The same ledger of 144 cells is summarized using equal-cell, dataset image-count, and annotated object-count weights, then after excluding each dataset and each corruption type in turn. These analyses change the descriptive target without requiring additional training, engine construction, or inference.

3.10. Architecture portability stress cases

We evaluate RT-DETR-L, a transformer detector that produces predictions directly [66, 67], and RetinaNet with a ResNet-50 feature pyramid, a dense detector that uses anchors [68, 69], on VOC, KITTI, and TT100K. Each detector family uses one checkpoint from a deterministic 100-epoch training run on each dataset. The compact portability records likewise do not preserve a separately auditable initialization field, so this stress test is interpreted at the level of the recorded trained checkpoints rather than a claim about a particular initialization recipe. Within a dataset, all treatments use the same fixed calibration set of 512 training images. We transfer the INT8 and FP8 construction procedures without tuning them for each family. The aim is to test whether the recorded procedure transfers to a different graph, not to estimate the performance of an optimized procedure for each family.

This extension contains 234 metric cells from original clean or corrupted sources and 18 separate clean cells encoded at JPEG quality 95. We reconstruct the main sign convention from the compact records:

$$E = (\mathrm{AP}_{\mathrm{FP32},c} - \mathrm{AP}_{q,c}) - (\mathrm{AP}_{\mathrm{FP32,J95}} - \mathrm{AP}_{q,\mathrm{J95}}). \quad (10)$$

The direct comparison remains $\Delta E = E_{\mathrm{INT8}} - E_{\mathrm{FP8}}$. Only point estimates are available, so the means and ranges are descriptive. We measure engine latency with three 30 second `trtexec` runs after a 5 second warmup. The measurement excludes preprocessing, transfer from the host, decoding, and NMS.

All manuscript summaries, tables, and figures are generated from fixed result ledgers using deterministic scripts. Before producing outputs, the research pipeline verifies expected factorial cell counts, paired components, sign conventions, low-AP counts, provenance records, and Secure Hash Algorithm 256 (SHA-256) bindings. The authors reran and reviewed these validations. The minimal journal-source archive contains publication sources; analysis scripts, compact ledgers, and audit reports are maintained in the versioned public research archive. This design follows recommendations emphasizing transparent experimental details and accessible artifacts [70].

OpenAI Codex assisted with refactoring manuscript artifact-generation and validation code. It did not train detectors, construct precision engines, or generate predictions. The authors inspected all code changes, reran assertion-based validators against retained ledgers, and independently checked the numerical outputs before inclusion. Other AI-assisted manuscript review is disclosed after the declarations section.

4. Results

4.1. Execution integrity and clean accuracy

All 12 FP16 consistency checks passed the 1.0-AP tolerance. The maximum FP32–FP16 difference was 0.540 AP on TT100K/YOLO11x and 0.253 AP among the area-evaluated COCO, VOC, and KITTI blocks. These checks exclude gross disagreement in the recorded TensorRT engines, not engine-build variation or exact source-checkpoint parity.

On clean images encoded at JPEG quality 95, FP8 achieved higher AP than INT8 in all 12 dataset–YOLO combinations. It outperformed INT8 by 1.40 AP points on average, with differences from 0.08 AP for TT100K/YOLO11x to 3.97 AP for KITTI/YOLO11m. This is the most consistent difference between the recorded treatments, but it does not show how their gap changes under corruption.

4.2. Absolute corrupted accuracy and the accuracy floor guard

Among the 288 corrupted YOLO treatment arms, 282 had lower AP than the corresponding JPEG-95 clean arm and six had small gains. The median signed clean-to-corrupted loss was 5.51 AP points, and the mean was 15.24 AP points. The larger mean reflects a minority of severe failures. Thirty-five arms fell below the descriptive guide of 10 AP, and nine fell below 5 AP. In the final rerun, 18 of 144 corrupted treatment arms fell below 10 AP and nine below 5 AP. These inventories show that a treatment gap can contract when absolute performance is poor; they are not statistically detected or application-specific failure thresholds.

Table 4 reports balanced summaries for each dataset and treatment. Means across all corruptions hide condition-level variation but clarify the interpretive boundary: ΔE describes a change in the treatment difference, not remaining task performance. The Supplementary Information reports size-endpoint AP and low-AP inventories.

4.3. Clean adjustment produces point-estimate sign reversals

The raw FP8–INT8 gap under corruption was positive in 134 of 144 cells and averaged 1.38 AP points. However, all 12 unique dataset–model clean gaps were also positive and averaged 1.40 AP points. Each clean gap supplies the baseline term for 12 corruption–severity cells and does not represent 12 independent clean experiments. After subtraction, mean ΔE was -0.02 AP, with 78 positive and 66 negative point estimates. Fifty-six cells changed from a positive raw gap to a negative adjusted point estimate (Fig. 2A); no reversal occurred in the opposite direction. Of these 56 point-estimate reversals, 17 adjusted intervals lay entirely below zero and 39 crossed zero. Moreover, 22 had $|\Delta E| < 0.25$ AP and 36 had $|\Delta E| < 0.50$ AP. Thus clean adjustment materially changes the question, but the count is not evidence for 56 multiplicity-adjusted discoveries.

Figure 2B compares the signed interaction with the lower corrupted AP of the two treatments. In 18 cells, at least one treatment falls below 10 AP. In five cells, at least one treatment falls below 5 AP. The 5- and 10-AP lines are descriptive diagnostic guides rather than universal deployment thresholds; an application-specific minimum should be fixed independently of the observed interaction. These cases show why removal of the clean gap and an absolute AP check answer different but related questions.

Table 4
Absolute accuracy check from the validated YOLO metric ledger. The loss is $D = \mathrm{AP}_{J95} - \mathrm{AP}_c$. Rows are balanced descriptive summaries and are not pooled leaderboard scores.

Dataset	Format	Clean AP	All-corrupt AP	Mean D	Severity-5 AP	Severity-5 D
COCO	INT8	46.010	34.911	11.099	25.185	20.826
COCO	FP8	47.110	35.824	11.286	25.667	21.443
VOC	INT8	65.732	49.246	16.486	35.478	30.255
VOC	FP8	66.828	50.300	16.529	36.023	30.805
KITTI	INT8	67.284	46.445	20.839	29.881	37.403
KITTI	FP8	69.797	48.498	21.299	31.504	38.293
TT100K	INT8	44.760	32.270	12.490	22.573	22.188
TT100K	FP8	45.636	33.752	11.884	23.467	22.169

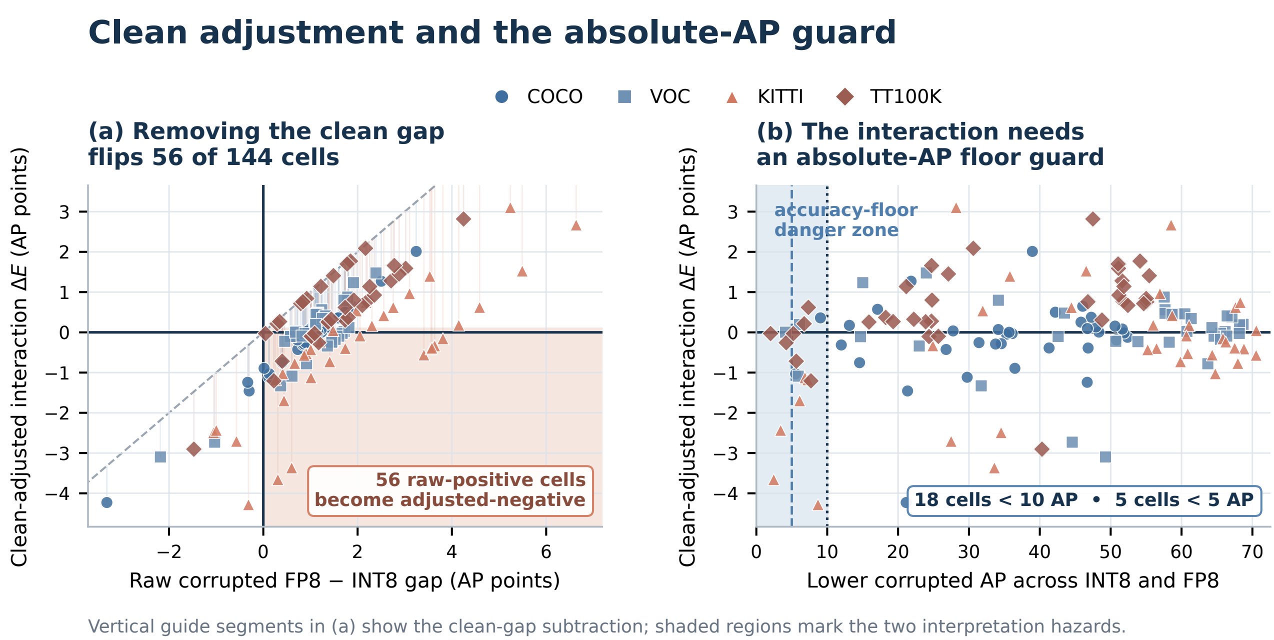

Figure 2: Effect of the protocol on point estimates. (A) The raw FP8–INT8 corrupted-input gap includes the matched-clean gap. Removing it produces 56 positive-to-negative point-estimate reversals; 17 adjusted intervals lie below zero and 39 cross zero. (B) Interaction values are plotted against the lower corrupted AP of the two treatments. Lines at 5 and 10 AP are descriptive guides, not universal deployment thresholds. Each point is one condition.

4.4. Exploratory interaction landscape

With common image bootstrap samples, the balanced value of $\overline{\Delta E}_{\mathrm{grid}}$ across the four datasets was -0.02 AP points. Its 95% percentile interval was $[-0.24, +0.15]$. The individual estimates varied widely. They ranged from -4.28 AP points for KITTI, YOLO11m, motion blur, and severity 5 to $+3.10$ AP points for KITTI, YOLO11n, fog, and severity 5. The dataset means also had opposite signs. TT100K had a mean of $+0.61$ AP points, while KITTI had a mean of -0.46 AP points (Table 5). The mean near zero therefore does not show that every condition has an interaction near zero.

For COCO, VOC, and KITTI, the separate area value of $\Delta\Psi$ was -0.50 AP points with interval $[-1.18, +0.01]$. For TT100K, the height-based value was -1.38 $[-2.51, -0.14]$. A negative value means that the INT8–FP8 interaction is more negative for the small endpoint than for the large endpoint; it does not establish useful small-object accuracy. Figure 3 presents all cells and joint endpoints, and Fig. 4 shows factor heterogeneity. The Supplementary Information gives dataset, factor, and size-endpoint summaries. Individual-cell intervals are exploratory and not multiplicity-adjusted.

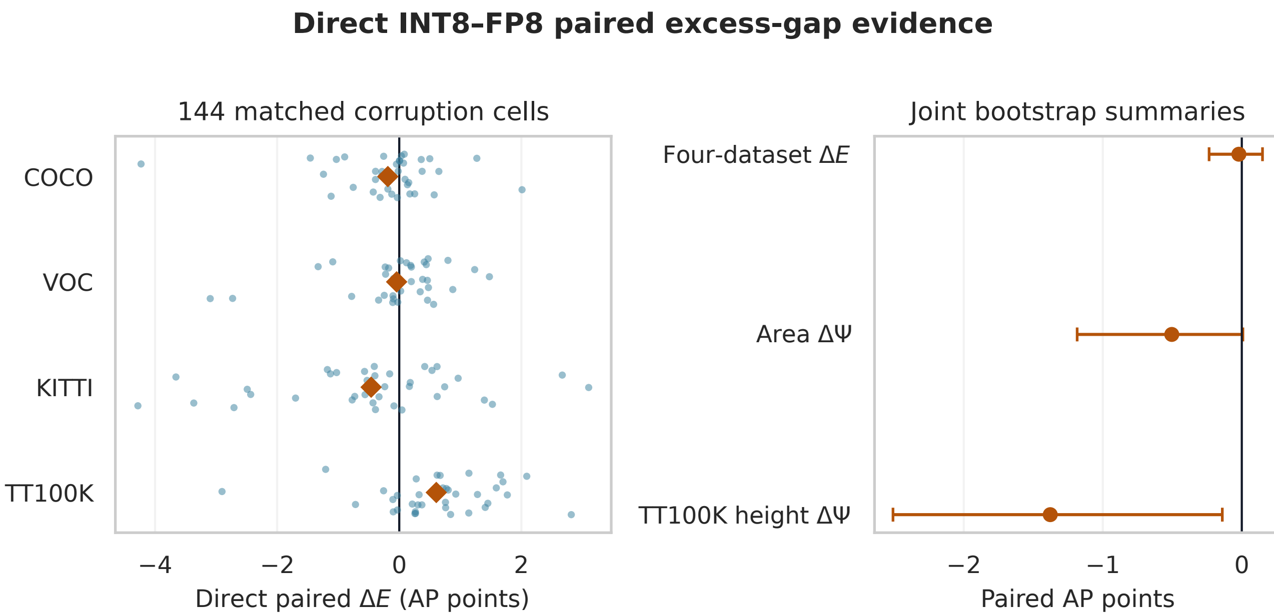

Figure 3: Exploratory evidence for the direct interaction. The left panel shows the estimates for all 144 cells and the dataset means. The right panel shows summaries from the paired bootstrap for the finite grid across four datasets and the two separate groups of size endpoints.

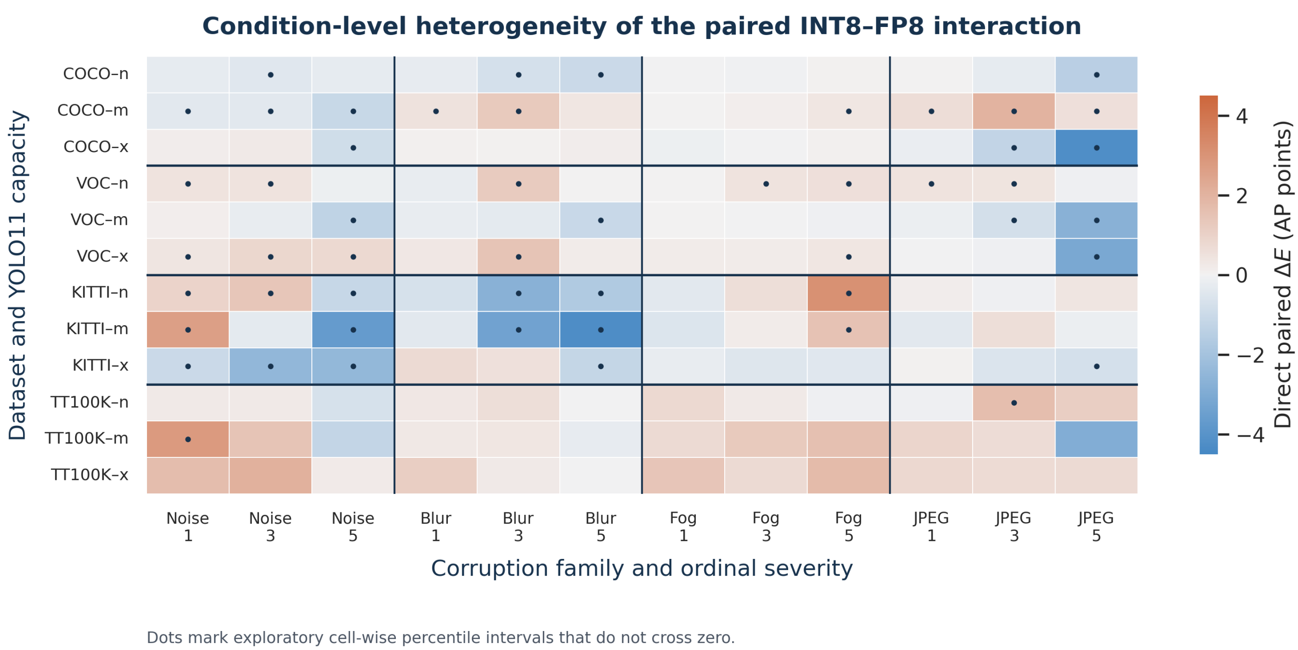

Figure 4: Variation across conditions. Columns group the corruption type and ordered severity, while rows group the dataset and YOLO11 model size. Dots mark exploratory percentile intervals for individual cells that do not cross zero. These intervals are not adjusted for multiple comparisons.

4.5. Evidence from final VOC and KITTI holdouts

The final rerun contains 72 cells with equal weights across datasets, models, corruptions, and severity levels. Its balanced direct interaction was -0.55 AP points. The 2.5th, 50th, and 97.5th percentiles were $[-0.78, -0.54, -0.29]$. The interaction was -0.43 AP points for VOC with an interval of $[-0.54, -0.29]$ and -0.67 AP points for KITTI with an interval of $[-1.13, -0.20]$ (Table 6). On average, FP8 retained 1.60 more clean AP points and had a positive clean advantage in all six combinations of dataset and

Table 5
Dataset summaries from the exploratory direct ledger. Values are AP points. TT100K uses the height endpoint, while the other datasets use the area endpoint.

Dataset	Size endpoint	Cells	Mean ΔE	Mean $\Delta\Psi$
COCO	area	36	-0.19	-0.12
VOC	area	36	-0.04	-0.44
KITTI	area	36	-0.46	-0.95
TT100K	height	36	+0.61	-1.38

model. The size interaction was -0.13 AP points with percentiles $[-0.98, -0.33, +0.38]$.

This rerun avoids reuse of the final images during checkpoint selection for these two datasets. All six strict reference-parity blocks passed; the largest recorded source-to-FP32-engine gap was 0.0404 AP and the largest FP32-to-FP16 gap was 0.2554 AP. The branch therefore supplies the paper's strongest post-selection holdout evidence, but its scope is narrower than the four-dataset exploratory grid. The two estimates answer different conditional questions and are not pooled as repeated estimates of one population parameter.

4.6. Sensitivity to the interaction scale

The additive and relative-retention contrasts answer related but non-equivalent questions (Table 7). Across the exploratory grid, mean ΔE was -0.02 AP while mean ΔR was $+0.75$ percentage points; their point-estimate signs differed in 18 of 144 cells. Across the final holdouts, the corresponding means were -0.55 AP and -0.12 percentage points, with seven sign differences. At dataset level, KITTI's final mean changed from -0.67 AP on the additive scale to $+0.10$ percentage points on the retention scale. Therefore no sign-only conclusion is stable without naming the estimand. We retain ΔE as primary because it directly describes a change in the between-treatment AP-point gap, while ΔR reports fractional performance retention analogous to clean-normalized corruption measures.

4.7. Sensitivity to weighting, realizations, seeds, and class universe

Table 8 makes the different targets of the four headline analyses explicit. Their intervals and descriptive summaries are conditional on different image universes and replication axes and must not be interpreted as repeated estimates of one parameter.

Table 6
Dataset summaries from the final holdouts. Effects and intervals are AP points from the common paired sampling schedule.

Dataset	Selection	Final	Cells	Mean ΔE	95% interval
VOC	2,510	5,823	36	-0.43	$[-0.54, -0.29]$
KITTI	1,496	1,197	36	-0.67	$[-1.13, -0.20]$

Table 7
Metric-scale sensitivity. ΔR is the FP8-minus-INT8 difference in the percentage of matched-clean AP retained. "Sign differences" counts cell-level disagreement between ΔE and ΔR.

Scope	Cells	Mean ΔE (AP)	Mean ΔR (pp)	Sign differences
Exploratory grid	144	-0.02	+0.75	18
Exploratory: COCO	36	-0.19	+0.16	1
Exploratory: VOC	36	-0.04	+0.35	6
Exploratory: KITTI	36	-0.46	+0.49	6
Exploratory: TT100K	36	+0.61	+1.98	5
Final holdouts	72	-0.55	-0.12	7
Final: VOC	36	-0.43	-0.34	3
Final: KITTI	36	-0.67	+0.10	4

Table 8
Scope of the principal conditionality analyses. "Cells" counts direct conditions; the realization row also states its 36 base conditions. S1/3/5 denote severity labels.

Analysis	Dataset/model/severity scope	Cells	Mean ΔE	Interval (AP)	Uncertainty target
Exploratory grid	4; n/m/x; S1/3/5	144	-0.02	[-0.24, +0.15]	Paired image bootstrap
Final holdouts	VOC/KITTI; n/m/x; S1/3/5	72	-0.55	[-0.78, -0.29]	Paired image bootstrap
Realization subset	3 datasets; m; S1/3/5	36 base / 108	-0.23	[-0.95, +0.37]	3-label nested sensitivity
Training/calibration seeds	3 datasets; m; S5	108	-1.11	not estimated	Descriptive 3×3 grid

Alternative aggregation methods did not produce a stable ranking of the recorded treatments. The means with equal weights, image count weights, and object count weights were -0.02, +0.00, and -0.09 AP points, respectively. The observed standard deviation (SD) across cells was 1.20 AP points. Means that omitted one dataset ranged from -0.23 to +0.13 AP points. Means that omitted one corruption type ranged from -0.16 to +0.05 AP points. Changes in sign near zero reflect variation across conditions rather than inconsistent calculations.

The three corruption materializations produced mean ΔE values of -0.12, -0.26, and -0.32 AP (SD 0.10); their balanced mean was -0.23, with nested sensitivity percentiles $[-0.95, -0.28, +0.37]$. For $\Delta\Psi$, the three means were -1.27, -0.83, and -1.01 AP (SD 0.22), with a balanced mean of -1.04 and nested percentiles $[-3.05, -1.42, +0.10]$. Across the stochastic corruptions only, mean between-realization variance was 0.5318 AP-points^2; deterministic JPEG is reported separately rather than contributing structural zeros to this mean. Because there are only three labels and this branch also changes image universe, capacity, dataset coverage, and severity nesting, these values are bounded design sensitivities rather than a population interval over corruptions.

Across the 108 targeted cells for training and calibration seeds, the mean ΔE with equal weights was -1.11 AP points. Marginal means for the training seeds ranged from -1.64 to -0.68 AP points. Marginal means for the calibration seeds ranged from -1.32 to -0.84 AP points. The combination of the original seeds produced -1.21 AP points, which was close to the balanced mean across the seed grid. However, the analysis covers only YOLO11m on three transfer datasets at severity 5.

For the TT100K stress cell with a fixed class set, 10,000 samples with positive weights gave ΔE percentiles of $[-1.19, -0.17, +0.88]$, versus $[-1.79, -0.20, +1.34]$ under the ordinary 2,000-draw bootstrap. The median sign and zero-crossing status were unchanged, and height $\Delta\Psi$ remained negative under both methods. This one-cell result does not validate the full 36-cell TT100K height macro. Named Supplementary sections provide the full available sensitivity tables.

Table 9
TensorRT engine measurements on the RTX 5090. Values for each condition are medians from three runs that were checked against the raw logs. Each row reports the median across 12 corresponding conditions.

Precision	Conditions	Median engine (MiB)	Median latency (ms)	Median throughput (qps)
FP32	12	79.90	2.434	410.6
INT8	12	24.07	0.789	1265.3
FP8	12	21.92	0.760	1315.7

4.8. Engine runtime

Across the 12 corresponding YOLO conditions, FP32 latency divided by INT8 latency had a median of 2.96× and a range from 1.49 to 3.14×. FP32 latency divided by FP8 latency had a median of 3.13× and a range from 1.78 to 3.36×. INT8 latency divided by FP8 latency had a median of 1.08× and ranged from 0.92 to 1.19×. FP8 was therefore not faster in every condition. These ratio summaries are medians of matched within-condition latency ratios and therefore need not equal ratios of the separate marginal medians reported in Table 9. Table 9 reports engine size, latency, and throughput. The measurements exclude image decoding, preprocessing, data transfer, detector decoding, NMS, power, and energy.

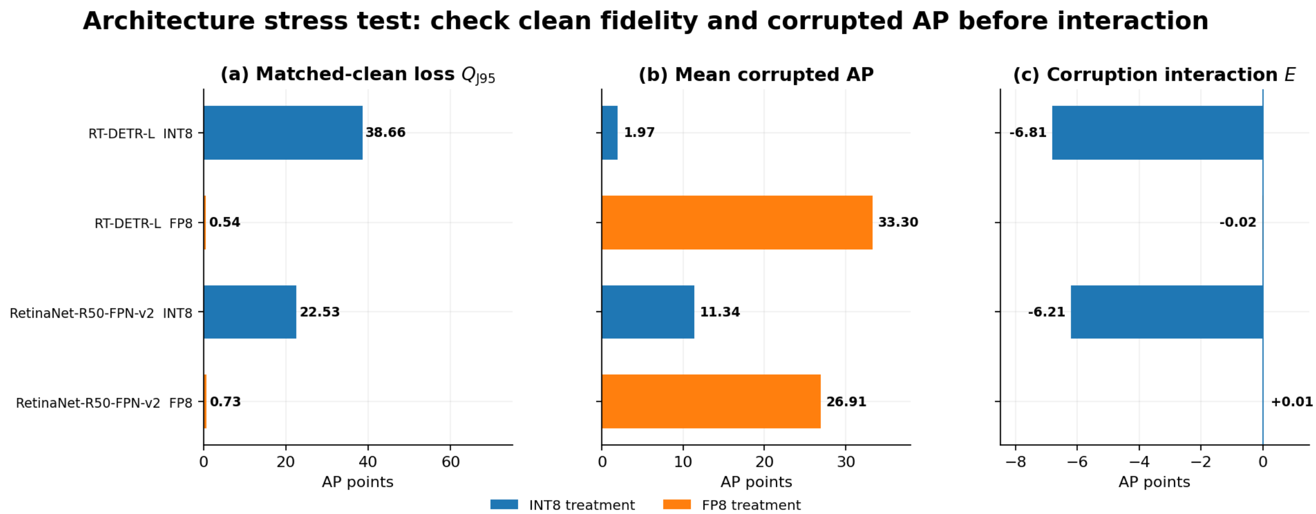

Figure 5: Recipe-portability stress tests across detector architectures. Matched-clean fidelity is checked first, absolute corrupted AP second, and interaction only third. Each bar gives equal weight to VOC, KITTI, and TT100K. The negative INT8 interaction follows a large clean loss and low corrupted AP and therefore does not demonstrate retained robustness.

4.9. Transfer tests across detector architectures

In all six combinations of detector family and dataset, FP8 remained within 1.15 clean AP points of the FP32 reference. The transferred INT8 procedure did not preserve clean accuracy. Its mean Q_{J95} was 38.66 AP points for RT-DETR-L and 22.53 AP points for RetinaNet. The largest losses were 69.30 AP points for VOC with RT-DETR-L and 43.30 AP points for KITTI with RetinaNet. Two FP32 transfer combinations also had low clean accuracy. RT-DETR-L on KITTI achieved 12.29 AP, and RetinaNet on TT100K achieved 3.37 AP. We retain these combinations to show transfer failure instead of removing them after inspection.

With the sign convention $E = Q_c - Q_{J95}$, the mean INT8 interaction was -6.81 AP points for RT-DETR-L and -6.21 AP points for RetinaNet. Mean corrupted INT8 AP was only 1.97 for RT-DETR-L and 11.34 for RetinaNet. AP was below 5 in 34 of 36 RT-DETR-L cells and 16 of 36 RetinaNet cells. The mean FP8 value of E was near zero, at -0.02 and $+0.01$ AP points, after much smaller clean gaps. Mean corrupted FP8 AP was 33.30 and 26.91. The descriptive direct interactions were -6.79 and -6.23 AP points. Figure 5 shows why these negative values do not demonstrate INT8 robustness. They mainly show that a large failure on clean images becomes smaller when absolute AP is already low.

Both quantized treatments reduced engine-only latency relative to the FP32 reference in every matched dataset–family condition. Per-dataset speedups ranged from 2.46–2.80× for RT-DETR INT8, 2.30–2.63× for RT-DETR FP8, 3.95–4.60× for RetinaNet INT8, and 3.38–3.94× for RetinaNet FP8. The lower latency of INT8 does not compensate for its clean-fidelity failure. The supported conclusion is that the fixed recipe did not transfer successfully, not that INT8 is inherently incompatible with either family.

5. Discussion

5.1. The protocol changes the question

The central methodological finding is that clean adjustment produced 56 positive-to-negative point-estimate reversals. When only corrupted-image performance is considered, FP8 outperforms INT8 in 134 of 144 cells. Yet FP8 also has higher AP in all 12 unique matched-clean comparisons that supply the repeated baseline terms. Once that pre-existing difference is removed, 66 interaction estimates are negative and the balanced additive mean approaches zero. The qualification is important: only 17 of the 56 adjusted intervals lie entirely below zero, whereas 39 cross zero, and many point effects are small. The diagnostic shows that the scientific question changes; it is not a count of confirmed treatment reversals.

The direct paired formulation keeps clean and corrupted outcomes for both treatments linked within each bootstrap image sample, preserving covariance induced by shared images, bytes, preprocessing, and decoding. Independently bootstrapping and then subtracting aggregated treatment effects would discard this pairing. FP32 also cancels from the direct interaction, so exploratory TF32 behavior cannot change ΔE, although it remains relevant to the separate reference-based Q and E diagnostics.

The comparison is nevertheless scale-dependent. Relative retention and additive AP points encode different notions of change, just as rPC and absolute corruption performance answer different within-model questions. Their signs disagree in 18 exploratory cells and seven holdout cells; KITTI's holdout dataset mean reverses sign. We therefore name the scale, retain additive ΔE as the primary between-treatment AP-gap estimand, and report ΔR as a sensitivity. Neither is a format-wide robustness score.

5.2. Clean accuracy, absolute accuracy, and interaction are distinct

The clean advantage of FP8, with a mean of 1.40 AP points, is consistent across all 12 YOLO combinations. This result is stronger and more uniform than the interaction under corruption. The exploratory grid has a mean of -0.02 AP points and an interval that crosses zero. In contrast, the final VOC and KITTI experiment has a mean of -0.55 AP points and an interval below zero. A negative ΔE means that the FP8–INT8 gap becomes smaller under corruption than on clean images. It does not mean that INT8 retains more useful accuracy under corruption or that FP8 has poor absolute robustness.

The absolute accuracy check is therefore necessary. AP decreases in 282 of 288 corrupted YOLO arms, and 35 arms fall below 10 AP. The transfer tests make the same point more clearly. The INT8 procedure transferred to RT-DETR begins with a mean clean gap of 38.66 AP points. It then produces a negative interaction while retaining only 1.97 mean AP under corruption. The interaction sign is mathematically correct, but it cannot support a deployment choice without the earlier checks of clean accuracy and low absolute corrupted AP. The evidence should be read in this order: clean accuracy, absolute corrupted accuracy, interaction, and then latency or engine size.

The floor guard has a limited purpose. We do not estimate a formal AP floor, correct for bounded outcomes, or threshold the primary measure. Instead, absolute AP and signed losses are reported beside the interaction. An application can define a minimum AP or maximum tolerated loss according to its risk tolerance, preferably before examining the interaction.

5.3. Why the exploratory and holdout estimates differ

The exploratory study and final VOC/KITTI experiment have different scopes. The broader grid includes COCO, TT100K, three model sizes, and partitions that were not all excluded from checkpoint selection; it characterizes a finite factorial design. The final experiment retrains six checkpoints and evaluates them after selection on disjoint final image lists. Its -0.55 AP estimate is the strongest holdout evidence in this study, but it does not cover COCO, TT100K, additional training seeds, or other detector families. KITTI's class-covering final subset is also a designed rather than random holdout, so its target distribution is conditional on that construction.

The difference between -0.02 and -0.55 AP should therefore not be treated as a failed replication. Dataset composition, partitions, retraining, and target scope all differ. This interpretation agrees with the dataset and omission summaries and the 1.20-AP cell SD. A future confirmatory study should preregister a deployment population, use untouched final partitions in every domain, and propagate training, calibration, engine-build, corruption, and image uncertainty in one hierarchical design.

5.4. Heterogeneity and object size

The grid mean near zero results from effects with opposite signs, not from a uniform absence of interaction. Dataset means range from -0.46 AP points on KITTI to +0.61 AP points on TT100K, and the individual estimates span more than seven AP points. Summaries with alternative weights and summaries that omit one factor remain small but cross zero. This pattern supports reporting results for each treatment and setting instead of one robustness rank for the recorded treatments.

The size endpoints also do not support a simple conclusion. The interval for $\Delta\Psi$ based on area reaches zero, while the TT100K endpoint based on height is negative. These quantities compare the treatment interaction for small and large objects. They do not measure the absolute robustness of small objects. AP for a specific size can be low, especially for TT100K objects in the XS group, so contraction near the accuracy floor can be stronger for the small endpoint. Area thresholds and thresholds based on the original box height also define different measurements and must not be pooled. The results therefore do not show that the interaction between quantization and corruption is always larger for small objects.

5.5. Interpretation of the complete INT8 and FP8 treatments

The graph audit shows that both nominal formats use mixed precision and that Q/DQ coverage depends on the architecture. Calibration, scale resolution, excluded operators, TensorRT runtime choices, output representation, and decoder behavior all affect the executable system. This view of the complete treatment helps explain why the same INT8 procedure can preserve YOLO accuracy but fail on RT-DETR or RetinaNet. The transfer test is useful because it reveals this failure. It does not provide a fair leaderboard across detector families.

The evidence does not identify a specific layer, quantizer, runtime stage, or decoder as the cause of the transferred INT8 recipe's failure. Diagnosis would require output-agreement checks, layer sensitivity, alternative calibration, selective higher precision, quantizer search, or quantization-aware training. The recorded FP8 treatment has small matched-clean gaps in all six transfer blocks, but this result depends on one procedure, one environment, one checkpoint per dataset–family block, one training seed, and one calibration list per dataset. Neither result establishes an inherent property of a number format.

5.6. Practical reporting recommendations

For a comparison of executable detectors under corruption, we recommend the following minimum report. First, define the corresponding clean image process and report the clean difference between treatments. Second, create each corrupted image independently of numerical precision and give the exact same bytes to all treatments. Third, estimate the adjusted interaction within common image samples. Fourth, report absolute corrupted AP or the loss from clean to corrupted images, and define any accuracy requirement for the application in advance. Fifth, retain results for individual conditions or defined groups so that a balanced mean does not hide changes in sign. Finally, describe the complete executable treatment and distinguish engine latency from total deployment latency.

These recommendations also apply beyond INT8 and FP8. The same four condition structure can compare quantizers, policies for mixed precision, compilers, calibration methods, or hardware runtimes whenever an existing clean difference may be confused with an interaction under a distribution change.

5.7. Limitations

Several limitations restrict the conclusions. The four synthetic transformations do not represent the full distribution of natural weather, sensor effects, compression artifacts, or domain changes, and severity gives only an order. Exploratory random materializations are not nested across severity. The three-realization branch is too small to estimate a population of possible fields or angles and simultaneously changes capacity, image universe, dataset coverage, and cross-severity dependence.

The exploratory summary is an arithmetic mean over a finite grid, not an effect weighted for a deployment population. The grid depends on one main training seed and recorded checkpoint-selection procedures. The final rerun covers only VOC and KITTI. The targeted seed analysis fixes YOLO11m, three transfer datasets, and severity 5, so it does not estimate full training or calibration variation. Architecture-portability checkpoints likewise use one seed. We did not repeat engine builds, and bootstrap intervals treat executable artifacts as fixed.

VOC and KITTI use annotations converted to COCO format rather than official challenge procedures, and KITTI's final subset is label-aware. The effective TT100K class set can change under standard image resampling; the fixed-universe analysis covers one stress cell and does not validate the full height macro. Area and height endpoints are not interchangeable. The historical exploratory reference may use TF32, whereas the holdout reference disables it; this distinction does not affect direct ΔE.

Runtime is measured on one RTX 5090 and TensorRT environment. It excludes preprocessing, data transfer, decoding, NMS, memory use, power, and energy. Counts of Q/DQ nodes in the graph do not reveal the precision of TensorRT kernels or execution outside the requested format. The compact package also lacks a complete software/build manifest and independently re-verifiable paired calibration identities for some older exploratory transfer engines. The transfer study across detector families uses point estimates, one training seed, one fixed calibration set per dataset, and procedures that were not optimized for each architecture. Finally, the compact submission package reproduces the manuscript summaries from fixed ledgers but does not distribute the datasets, checkpoints, engines, or raw predictions.

6. Conclusion

This study addressed a central ambiguity in robustness comparisons between quantized object detectors: a difference measured only on corrupted images combines the accuracy gap already present on clean inputs with the additional change observed under corruption. We proposed a paired, floor-guarded evaluation protocol that supplies identical image bytes to the INT8 and FP8 TensorRT treatments, removes their corresponding clean-image difference, and preserves pairing through common-image bootstrap samples. The resulting interaction is interpreted jointly with clean AP and absolute corrupted AP, ensuring that a reduced gap between treatments near an accuracy floor is not mistaken for retained detection performance.

Across the 144-cell exploratory grid, FP8 outperformed INT8 by 1.40 matched-clean AP points, while balanced ΔE was -0.02 $[-0.24, +0.15]$. Post-selection evaluation on final VOC/KITTI images excluded from checkpoint selection yielded -0.55 $[-0.78, -0.29]$. Clean adjustment produced 56 point-estimate sign reversals, but only 17 adjusted intervals lay entirely below zero. Relative-retention signs differed from additive signs in 18 cells, demonstrating scale dependence. Meanwhile, AP declined in 282 of 288 corrupted arms, so treatment-gap contraction cannot substitute for absolute accuracy.

The results do not establish a universal robustness ranking. Clean fidelity, absolute corrupted performance, and a scale-explicit change in the treatment gap are distinct questions. Reporting them jointly for complete executable treatments provides a transparent, internally traceable, and decision-relevant basis for evaluating quantized detectors under distribution degradation.

Data and code availability

The public repository provides analysis code, compact CSV/JSON ledgers, validation reports, and deterministic scripts for regenerating reported summaries and figures. The journal-source archive contains only publication sources. Raw datasets are not redistributed; checkpoints, TensorRT engines, and raw predictions are also omitted because of storage and portability constraints. Retained records bind summaries to external artifacts with SHA-256 hashes, supporting summary-level recomputation and internal provenance checks rather than bit-exact regeneration of every inference artifact. The versioned archive is maintained under concept DOI `https://doi.org/10.5281/zenodo.22031663`, and the repository is `https://github.com/iamdinhthuan/paired-floor-guarded-int8-fp8-detection`. The version DOI for this CVIU revision will be recorded in the submission metadata after its release is published.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Dinh Thuan Nguyen: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing–original draft. **Lam Phuong Nguyen:** Validation, Data curation, Writing–review and editing. **Vinh Huy Nguyen:** Methodology, Visualization, Writing–review and editing. **Sy Vu Quang:** Validation, Writing–review and editing. **Mohan Rajesh Elara:** Supervision, Writing–review and editing. **Anh Vu Le:** Supervision, Project administration, Writing–review and editing.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During preparation of this work, the authors used OpenAI Codex for code refactoring, validation support, consistency checks, and language editing; OpenAI ChatGPT for literature-discovery support and manuscript critique; and Anthropic Claude Code for independent manuscript critique and submission-file review. These systems were not listed as authors or contributors and did not autonomously train detectors, build engines, generate predictions, fabricate or alter research data, or make final scientific decisions. The authors verified cited sources and numerical claims, reviewed every retained change, reran relevant validation scripts, and take full responsibility for the article.

References

[1] L. Wen, D. Du, Z. Cai, Z. Lei, M.-C. Chang, H. Qi, J. Lim, M.-H. Yang, S. Lyu, UA-DETRAC: A new benchmark and protocol for multi-object detection and tracking, Computer Vision and Image Understanding 193 (2020) 102907. doi:10.1016/j.cviu.2020.102907.

[2] R. Krishnamoorthi, Quantizing deep convolutional networks for efficient inference: A whitepaper, arXiv preprint arXiv:1806.08342 (2018). doi:10.48550/arXiv.1806.08342.

[3] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 2704–2713. doi:10.1109/CVPR.2018.00286.

[4] R. Banner, Y. Nahshan, D. Soudry, Post training 4-bit quantization of convolutional networks for rapid-deployment, in: Advances in Neural Information Processing Systems, Vol. 32, 2019, pp. 7948–7956.

[5] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, D. S. Modha, Learned step size quantization, in: International Conference on Learning Representations, 2020. URL https://openreview.net/forum?id=rkgO66VKDS

[6] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. W. Mahoney, K. Keutzer, HAWQ-V3: Dyadic neural network quantization, in: Proceedings of the 38th International Conference on Machine Learning, Vol. 139 of Proceedings of Machine Learning Research, 2021, pp. 11875–11886.

[7] N. Wang, J. Choi, D. Brand, C.-Y. Chen, K. Gopalakrishnan, Training deep neural networks with 8-bit floating point numbers, in: Advances in Neural Information Processing Systems, Vol. 31, 2018.

[8] X. Sun, J. Choi, C.-Y. Chen, N. Wang, S. Venkataramani, V. Srinivasan, X. Cui, W. Zhang, K. Gopalakrishnan, Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks, in: Advances in Neural Information Processing Systems, Vol. 32, 2019.

[9] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, H. Wu, FP8 formats for deep learning, arXiv preprint arXiv:2209.05433 (2022). doi:10.48550/arXiv.2209.05433.

[10] H. Shen, N. Mellempudi, X. He, Q. Gao, C. Wang, M. Wang, Efficient post-training quantization with FP8 formats, in: Proceedings of Machine Learning and Systems, Vol. 6, 2024, pp. 483–498.

[11] D. Hendrycks, T. Dietterich, Benchmarking neural network robustness to common corruptions and perturbations, in: International Conference on Learning Representations, 2019.

[12] C. Kamann, C. Rother, Benchmarking the robustness of semantic segmentation models, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 8828–8838.

[13] C. Michaelis, M. Mitzkus, R. Geirhos, E. Rusak, O. Bringmann, A. S. Ecker, M. Bethge, W. Brendel, Benchmarking robustness

in object detection: Autonomous driving when winter is coming, arXiv preprint arXiv:1907.07484 (2019). doi:10.48550/arXiv.1907.07484.

[14] J. Liu, Z. Wang, L. Ma, C. Fang, T. Bai, X. Zhang, J. Liu, Z. Chen, Benchmarking object detection robustness against real-world corruptions, International Journal of Computer Vision 132 (10) (2024) 4398–4416. doi:10.1007/s11263-024-02096-6.

[15] M. Nagel, M. van Baalen, T. Blankevoort, M. Welling, Data-free quantization through weight equalization and bias correction, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 1325–1334.

[16] Y. Cai, Z. Yao, Z. Dong, A. Gholami, M. W. Mahoney, K. Keutzer, ZeroQ: A novel zero shot quantization framework, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 13169–13178.

[17] M. Nagel, R. A. Amjad, M. van Baalen, C. Louizos, T. Blankevoort, Up or down? adaptive rounding for post-training quantization, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, 2020, pp. 7197–7206.

[18] I. Hubara, Y. Nahshan, Y. Hanani, R. Banner, D. Soudry, Accurate post training quantization with small calibration sets, in: Proceedings of the 38th International Conference on Machine Learning, Vol. 139 of Proceedings of Machine Learning Research, 2021, pp. 4466–4475.

[19] Y. Li, R. Gong, X. Tan, Y. Yang, P. Hu, Q. Zhang, F. Yu, W. Wang, S. Gu, BRECQ: Pushing the limit of post-training quantization by block reconstruction, in: International Conference on Learning Representations, 2021.

[20] X. Wei, R. Gong, Y. Li, X. Liu, F. Yu, QDrop: Randomly dropping quantization for extremely low-bit post-training quantization, in: International Conference on Learning Representations, 2022.
URL https://openreview.net/forum?id=ySQH0oDyp7

[21] Z. Yuan, C. Xue, Y. Chen, Q. Wu, G. Sun, PTQ4ViT: Post-training quantization for vision transformers with twin uniform quantization, in: Computer Vision–ECCV 2022, Vol. 13672 of Lecture Notes in Computer Science, 2022, pp. 191–207. doi:10.1007/978-3-031-19775-8_12.

[22] Y. Lin, T. Zhang, P. Sun, Z. Li, S. Zhou, FQ-ViT: Post-training quantization for fully quantized vision transformer, in: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, 2022, pp. 1173–1179. doi:10.24963/ijcai.2022/164.

[23] Z. Li, J. Xiao, L. Yang, Q. Gu, RepQ-ViT: Scale reparameterization for post-training quantization of vision transformers, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 17227–17236.

[24] Z. Li, Q. Gu, I-ViT: Integer-only quantization for efficient vision transformer inference, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 17065–17075.

[25] Y. Wei, X. Pan, H. Qin, W. Ouyang, J. Yan, Quantization mimic: Towards very tiny CNN for object detection, in: Computer Vision–ECCV 2018, 2018, pp. 267–283.

[26] R. Li, Y. Wang, F. Liang, H. Qin, J. Yan, Fully quantized network for object detection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 2810–2819.

[27] P. Chen, J. Liu, B. Zhuang, M. Tan, C. Shen, AQD: Towards accurate quantized object detection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 104–113.

[28] J. Liu, L. Niu, Z. Yuan, D. Yang, X. Wang, W. Liu, PD-Quant: Post-training quantization based on prediction difference metric, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 24427–24437. doi:10.1109/CVPR52729.2023.02340.

[29] L. Niu, J. Liu, Z. Yuan, D. Yang, X. Wang, W. Liu, Improving post-training quantization on object detection with task loss-guided L_p metric, arXiv preprint arXiv:2304.09785 (2023). doi:10.48550/arXiv.2304.09785.

[30] Y. Ding, W. Feng, C. Chen, J. Guo, X. Liu, Reg-PTQ: Regression-specialized post-training quantization for fully quantized object detector, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 16174–16184.

[31] M. Kim, D. Lee, J. Yu, J. Hur, G. Kim, J. Kim, Inlier-centric post-training quantization for object detection models, in: International Conference on Learning Representations, 2026.
URL https://openreview.net/forum?id=GN9otzf5o6

[32] M. Bonnaerens, M. Freiberger, J. Dambre, Anchor pruning for object detection, Computer Vision and Image Understanding 221 (2022) 103445. doi:10.1016/j.cviu.2022.103445.

[33] S. N. Tien, T. L. Tien, H. L. Thanh, N. L. Tu, ONNX-based architectures for post-training quantization face detection on edge devices, Advances in Electrical and Electronic Engineering, Early Access (2026).
URL https://advances.vsb.cz/early-access

[34] B. Noune, P. Jones, D. Justus, D. Masters, C. Luschi, 8-bit numerical formats for deep neural networks, arXiv preprint arXiv:2206.02915 (2022). doi:10.48550/arXiv.2206.02915.

[35] A. Kuzmin, M. van Baalen, Y. Ren, M. Nagel, J. Peters, T. Blankevoort, FP8 quantization: The power of the exponent, in: Advances in Neural Information Processing Systems, Vol. 35, 2022. doi:10.52202/068431-1065.

[36] Z. Zhang, Y. Zhang, G. Shi, Y. Shen, R. Gong, X. Xia, Q. Zhang, L. Lu, X. Liu, Exploring the potential of flexible 8-bit format: Design and algorithm, arXiv preprint arXiv:2310.13513 (2023). doi:10.48550/arXiv.2310.13513.

[37] M. van Baalen, A. Kuzmin, S. S. Nair, Y. Ren, E. Mahurin, C. Patel, S. Subramanian, S. Lee, M. Nagel, J. Soriaga, T. Blankevoort, FP8 versus INT8 for efficient deep learning inference, arXiv preprint arXiv:2303.17951 (2023). doi:10.48550/arXiv.2303.17951.

[38] E. Mintun, A. Kirillov, S. Xie, On interaction between augmentations and corruptions in natural corruption robustness, in: Advances in Neural Information Processing Systems, Vol. 34, 2021.

[39] J. Cheng, B. Huang, Y. Fang, Z. Han, Z. Wang, Adversarial intensity awareness for robust object detection, Computer Vision and Image Understanding 251 (2025) 104252. doi:10.1016/j.cviu.2024.104252.

[40] D. Hoiem, Y. Chodpathumwan, Q. Dai, Diagnosing error in object detectors, in: Computer Vision–ECCV 2012, 2012, pp. 340–353. doi:10.1007/978-3-642-33712-3_25.

[41] D. Bolya, S. Foley, J. Hays, J. Hoffman, TIDE: A general toolbox for identifying object detection errors, in: Computer Vision–ECCV 2020, 2020, pp. 558–573. doi:10.1007/978-3-030-58580-8_33.

[42] K. Oksuz, B. C. Cam, S. Kalkan, E. Akbas, Imbalance problems in object detection: A review, IEEE Transactions on Pattern Analysis and Machine Intelligence 43 (10) (2021) 3388–3415. doi:10.1109/TPAMI.2020.2981890.

[43] P. Luo, J. Nie, J. Xie, J. Cao, X. Zhang, Localization-aware logit mimicking for object detection in adverse weather conditions, Image and Vision Computing 146 (2024) 105035. doi:10.1016/j.imavis.2024.105035.

[44] S. Zhang, L. Zhang, Z. Liu, Test-time adaptation for object detection via Dynamic Dual Teaching, Image and Vision Computing 163 (2025) 105740. doi:10.1016/j.imavis.2025.105740.

[45] K. Su, Y. Tomioka, Q. Zhao, Y. Liu, YOLIC: An efficient method for object localization and classification on edge devices, Image and Vision Computing 147 (2024) 105095. doi:10.1016/j.imavis.2024.105095.

[46] X. Zheng, H. Wang, Y. Shang, G. Chen, S. Zou, Q. Yuan, Starting from the structure: A review of small object detection based on deep learning, Image and Vision Computing 146 (2024) 105054. doi:10.1016/j.imavis.2024.105054.

[47] J. Liu, Y. Xie, J. Wang, A lightweight and robust framework for small object detection in UAV imagery, Computer Vision and Image Understanding 267 (2026) 104735. doi:10.1016/j.cviu.2026.104735.

[48] G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng, X. Xie, J. Han, Towards large-scale small object detection: Survey and benchmarks, IEEE Transactions on Pattern Analysis and Machine Intelligence 45 (11) (2023) 13467–13488. doi:10.1109/TPAMI.2023.3290594.

[49] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, T. Darrell, Tent: Fully test-time adaptation by entropy minimization, in: International Conference on Learning Representations, 2021. URL https://openreview.net/forum?id=uXl3bZLkr3c

[50] Q. Wang, O. Fink, L. Van Gool, D. Dai, Continual test-time domain adaptation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 7201–7211.

[51] S. Niu, J. Wu, Y. Zhang, Y. Chen, S. Zheng, P. Zhao, M. Tan, Efficient test-time model adaptation without forgetting, in: Proceedings of the 39th International Conference on Machine Learning, Vol. 162 of Proceedings of Machine Learning Research, 2022, pp. 16888–16905.

[52] T. Karimov, H. Imani, A. Kazakov, Quantization robustness to input degradations for object detection, in: 2025 Innovations in Intelligent Systems and Applications Conference (ASYU), IEEE, 2025. doi:10.1109/ASYU67174.2025.11208310.

[53] J. Xiao, Z. Li, L. Yang, Y. Mei, Q. Gu, TTAQ: Towards stable post-training quantization in continuous domain adaptation, arXiv preprint arXiv:2412.09899 (2024). doi:10.48550/arXiv.2412.09899.

[54] H. Yaghoubi Araghi, P. Pilevar, M. C. Lin, Recti-Q: Feature-space rectification for out-of-distribution-robust quantized perception in edge robotics, arXiv preprint arXiv:2607.18540 (2026). doi:10.48550/arXiv.2607.18540.

[55] Q. Li, Y. Meng, C. Tang, J. Jiang, Z. Wang, Investigating the impact of quantization on adversarial robustness, arXiv preprint arXiv:2404.05639 (2024). doi:10.48550/arXiv.2404.05639.

[56] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick, Microsoft COCO: Common objects in context, in: Computer Vision–ECCV 2014, 2014, pp. 740–755. doi:10.1007/978-3-319-10602-1_48.

[57] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman, The PASCAL visual object classes challenge: A retrospective, International Journal of Computer Vision 111 (1) (2015) 98–136. doi:10.1007/s11263-014-0733-5.

[58] A. Geiger, P. Lenz, R. Urtasun, Are we ready for autonomous driving? the KITTI vision benchmark suite, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2012, pp. 3354–3361. doi:10.1109/CVPR.2012.6248074.

[59] Z. Zhu, D. Liang, S. Zhang, X. Huang, B. Li, S. Hu, Traffic-sign detection and classification in the wild, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 2110–2118.

[60] G. Jocher, J. Qiu, Ultralytics YOLO11, official software documentation and citation record; accessed 14 August 2026 (2024). URL https://docs.ultralytics.com/models/yolo11/

[61] NVIDIA Corporation, NVIDIA TensorRT 11.1.0 release notes, Official software documentation, the recorded build reports version 11.1.0.106; accessed 3 September 2026 (2026). URL https://docs.nvidia.com/deeplearning/tensorrt/latest/getting-started/release-notes-11/11.1.0.html

[62] NVIDIA Corporation, NVIDIA Model Optimizer 0.45.0 release, Official software release, version 0.45.0; accessed 22 August 2026 (2026). URL https://github.com/NVIDIA/Model-Optimizer/releases/tag/0.45.0

[63] B. Efron, Bootstrap methods: Another look at the jackknife, The Annals of Statistics 7 (1) (1979) 1–26. doi:10.1214/aos/1176344552.

[64] D. B. Rubin, The bayesian bootstrap, The Annals of Statistics 9 (1) (1981) 130–134. doi:10.1214/aos/1176345338.

[65] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Mohammadi Sepahvand, E. Raff, K. Madan, V. Voleti, S. Ebrahimi Kahou, V. Michalski, T. Arbel, C. Pal, G. Varoquaux, P. Vincent, Accounting for variance in machine learning benchmarks, in: Proceedings of Machine Learning and Systems, Vol. 3, 2021.

[66] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko, End-to-end object detection with transformers, in: Computer Vision–ECCV 2020, 2020, pp. 213–229. doi:10.1007/978-3-030-58452-8_13.

[67] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, J. Chen, DETRs beat YOLOs on real-time object detection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 16965–16974.

[68] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 2980–2988. doi:10.1109/ICCV.2017.324.

[69] PyTorch Core Team, Torchvision 0.28 RetinaNet ResNet-50 FPN V2 documentation, Official software documentation, version 0.28; accessed 22 August 2026 (2026). URL https://docs.pytorch.org/vision/0.28/models/generated/torchvision.models.detection.retinanet_resnet50_fpn_v2.html

[70] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Lariviere, A. Beygelzimer, F. d'Alche Buc, E. Fox, H. Larochelle, Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program, Journal of Machine Learning Research 22 (164) (2021) 1–20.